



Codex 3P Orchestrator · 지도교수 지도편달 자문서

3대 AI 병렬 협업 체계 현황, 아키텍처 실증 결과 및 직면 난제에 대한 공학적 조언 요청

수신: 지도교수님

작성: Codex 사령부 (with Claude & Antigravity)

v2.0 CSC 수직 슬라이스 실증

Windows Native 제약

✦ 교수님 피드백 반영 경과 및 시스템 전환 (Pivot) 보고

지난 카카오톡 조언을 바탕으로 한 아키텍처 체질 개선 결과

💡 교수님의 핵심 조언 수용 결과 (패러다임 대전환)

교수님께서 지적해주신 "DB가 기억하고, 코드가 조율하며, 코드가 검증한다. AI는 `--ephemeral` (단발성)로 매번 새로 태어나는 순수 함수다"라는 대원칙을 전면 채택했습니다.

이에 따라 비결정적인 LLM 내부 상태에 의존하던 기존의 장기 세션(Long-lived Session) 모델에서 스케줄러 코드 중심의 결정론적 파이프라인으로 피벗(Pivot)을 완료했습니다.

📊 설계 전제 6대 맹점 개선 대조표

#	이전 설계 (우리들의 맹점)	교수님 제시 구조	조치 결과 및 현재 상태
1	AI가 장기 세션을 유지하며 대화	<code>--ephemeral</code> 단발성 실행	<code>csc_slice.py</code> 를 통해 1회성 CLI 호출 및 세션 소실·락 고착 원천 소멸 실증
2	Codex(LLM)가 뇌로서 전체 조율	JS/Python 스케줄러 코드가 조율	스케줄러가 JSON 스키마를 강제하고 AI는 단순 <code>Provider</code> 로 격하
3	AI가 자기 성공과 종료코드를 증명	코드가 직접 실행해 실측	<code>evidence_gate()</code> 구현: AI가 거짓 주장해도 코드가 실제 종료코드로 뒤엎음
4	로컬 상주 프로세스 불가 판정	PM2 등의 프로세스 매니저	현재 순수 Python 백그라운드 관리와 PM2 도입 간 저울질 중
5	안전을 마크다운 규칙으로 강제	<code>--sandbox read-only</code> 프로세스 격리	쓰기 불가 상태로 자식 프로세스를 기동하여 보안 원천 차단
6	마크다운 파일 기반 거버넌스	SQLite 데이터베이스 트랜잭션	<code>slice.db</code> 를 통한 멍등성(Idempotency) 및 호출 이력 구조화 저장

수직 슬라이스 (Vertical Slice) 자체 실증 결과

외부 의존성을 배제한 순수 코드 검증 9/9 통과 및 실제 모델 호출 실측

csc_slice.py 자체검증 9/9 전원 통과

비정상 입력 및 AI의 거짓말 시나리오를 코드가 방어하는지 전수 검증했습니다:

- ✓ **[OK] valid**: 정상 스키마 JSON은 안전하게 DB 저장
- ✓ **[OK] bad_enum**: 허용되지 않은 임의 판정값 거부
- ✓ **[OK] out_of_range**: 신뢰도 수치(0.0~1.0) 초과 거부
- ✓ **[OK] missing / extra**: 필수 키 누락 및 위조 여분 키 배제
- ✓ **[OK] liar**: 형식은 맞으나 진위는 미확인된 경우 분리 기록
- ✓ **[OK] idempotent**: 동일 요청 재실행 시 중복 적재 방지
- 🔥 **[OK] override**: AI 주장=GO, 실제 종료코드=1 → 코드가 BLOCKED로 역전!
- ✓ **[OK] confirm**: AI 주장=GO, 실제 종료코드=0 → 코드가 최종 GO 인가

실제 모델 CLI 호출 실측 증거

교수님의 원칙대로 Claude Code CLI를 단발성 서버프로세스로 기동하여 결과를 수집했습니다:

```
$ python csc_slice.py run --provider
claude --task "1+1의 값이 2인지 판정하
라."

{
  "call_id": "4364e8fcdfa6541d",
  "status": "STORED",
  "data": {
    "verdict": "GO",
    "confidence": 0.99,
    "reason": "표준 산술에서 1+1은 2이
므로 참이다."
  },
  "elapsed_s": 8.07
}
```

* Codex CLI의 경우 명령줄 해석(.CMD vs .EXE) 및 토큰 소진 이슈를 확인하여 핸들러를 보강했습니다.

⚠ 현재 프로젝트가 직면한 4대 공학적 난제 (Engineering Bottlenecks)

교수님의 현장 노하우와 실전 경험에 입각한 조언이 절실한 부분입니다

난제 1. Windows 파일 잠금 핸들 경합 (WinError 5 PermissionError)

현상: 워커 메타데이터 갱신 시 원자적 쓰기(`_atomic_write_json` → `os.replace`)를 수행할 때, Windows의 파일 시스템 인덱서, 안티바이러스, 또는 비동기 리더가 일시적으로 핸들을 쥐고 있어 `[WinError 5] 액세스가 거부되었습니다` 에러가 발생하며 백그라운드 프로세스가 종료되는 현상.

시도한 대책: 최대 5회 재시도(0.02s 간격). 그러나 순간적 파일 잠금 시 여전히 실패하여 프로세스 생존성이 떨어짐.

난제 2. 소켓 브로커(Real-time Mesh) vs Ephemeral CLI Provider 통합 딜레마

현상: 본선 시스템은 실시간 소켓 브로커(`csc_broker.py`, 127.0.0.1 TCP)를 두어 세 도구 간 빠른 이벤트 교환을 꾀했으나, 교수님께서 제안하신 `--ephemeral` 모델(필요할 때만 CLI를 잠깐 띄워 호출하고 종료)과 결합하려 할 때 "상주 브로커가 굳이 필요한가?" vs "완전 단발성 실행 시 CLI 기동 오버헤드(각 5~8초)가 누적되어 상호작용 속도가 너무 느려지는 문제"가 충돌함.

난제 3. 로컬 런타임 프로세스 관리자(Process Supervisor) 선택 문제

현상: 교수님께서서는 PM2를 추천해주셨으나, 본 워크스페이스는 개발자 일반 환경(Windows)에 배포될 예정입니다. Node.js/PM2를 전제하면 비개발자 또는 순수 Python 환경에서의 설치 복잡도와 의존성이 증가합니다. 순수 Python 데몬으로 유지할지, PM2나 공식 러너를 강제할지에 대한 판단이 필요합니다.

난제 4. LLM 비결정성 제어와 '거짓 성공(Hallucination)' 차단 및 완전성

현상: 스키마 검증(Pydantic/JSON Schema)은 형식만 잡고, 실제 거짓말(예: 존재하지 않는 커밋 해시 주장, 미실행 테스트 통과 주장)은 통과합니다. `evidence_gate` 로 명령어를 직접 실행해 검증하는 방식을 도입했으나, 실행 대상 명령어 자체가 다양해질 때 검증 비용과 범위를 어디까지 엄격하게 둘 것인가의 문제입니다.

⚖️ 향후 진행 방향에 대한 아키텍처 대안 3종 비교

각 대안의 기술적 부채, 실현 가능성, 유지보수성 평가

구분	대안 A. 순수 Ephemeral 스케줄러 (교수님 원안)	대안 B. 상주 소켓 브로커 + 데몬 (현재 본선)	대안 C. 하이브리드
구조	스케줄러가 매번 <code>subprocess.run</code> 으로 CLI 단 발 호출, SQLite에만 기록	127.0.0.1 TCP 소켓 서버 상주, 세 도구 워커 가 TCP 연결 유지	상태/결정은 SC 주도하되, 실시 푸시만 경량 로
장점	세션 소실, 동기화 락, 좀비 프로세스 문제가 100% 소멸. 가장 단순하고 견고함.	초고속 메시지 중계 (지연시간 < 5ms). 에이 전트 간 즉각적 팽풍 토론 가능.	스케줄러의 결 실시간 사용자 을 동시에 확보
단점	매 턴마다 CLI 기동 오버헤드 (5~10초 소요). 도 구 간 연속 토론 시 지연 큼.	Windows 프로세스/파일 잠금 문제 취약. 브 로커 생존 관리(Liveness) 오버헤드 큼.	두 메커니즘을 므로 코드 복잡
추천 도	★★★★ (안정성 최상)	★★ (운영 리스크 존재)	★★★★★

지도교수님께 청하는 4대 핵심 지도편달 질의

바쁘신 일정 중에도 아래 4가지 핵심 질문에 대해 고견을 나누어 주시면 감사하겠습니다

Q1. 프로세스 생명주기

중요도: High

Q1. CLI 기동 지연(5~8초)을 감수하고서라도 순수 Ephemeral로 완전히 전환하는 것이 프로덕션 관점에서 더 우월할까요?

상주 소켓 브로커를 두면 반응 속도는 빠르지만 Windows 파일 핸들 경합과 프로세스 관리 비용이 발생합니다. 반면 완전 단발성(Ephemeral)은 무결성은 완벽하지만 사용자 체감 속도가 떨어집니다.

[선택지 A] 속도보다 신뢰성. 10초가 걸리더라도 순수 Ephemeral + SQLite 구조로 단일화하라.

[선택지 B] 하이브리드 유지. 비즈니스 판단은 Ephemeral로 하되, 가벼운 알림 통신은 백그라운드 브로커를 보강하라.

Q2. 런타임 매니저

중요도: High

Q2. Windows 로컬 환경에서 PM2 도입을 필수 전제로 삼아야 할까요, 아니면 순수 Python 관리자로 충분할까요?

교수님 실사용 구조에서는 PM2를 사용하셨는데, 일반 사용자 로컬 PC에 Node.js + PM2를 요구하는 것이 배포 허들이 되지 않을까 고민입니다.

[선택지 A] PM2가 표준이다. Node.js 의존성을 두더라도 프로세스 부활과 로그 관리를 PM2에 일임하라.

[선택지 B] Python 네이티브(asyncio subprocess + Windows Service/경량 위치)로 의존성 없이 자립시켜라.

Q3. Windows 파일 잠금

중요도: Medium

Q3. Windows os.replace 시 발생하는 [WinError 5] PermissionError를 방어하는 가장 우아한 방법은 무엇일까요?

원자적 교체 시 타 프로세스의 순간 점유로 실패합니다. 지수 백오프(Exponential Backoff) 외에 Windows 전용 API(MoveFileExW REPLACE_EXISTING)나 SQLite WAL 모드로 파일 교체 자체를 배제하는 방안 중 어떤 쪽을 추천하시나요?

Q4. AI의 거짓말(Hallucination)을 막는 evidence_gate의 실측 범위를 어디까지 넓혀야 할까요?

현재는 단위테스트 종료코드(Exit Code 0), Git 커밋 해시 일치, 파일 실제 존재 여부를 코드가 직접 확인합니다. 이 외에 프로덕션에서 반드시 강제해야 할 추가 실측 검증 항목이 있을까요?